Lecture 15: Shortest Paths in Practice

Topics

- Weighted graphs and edge weights in real networks
- BFS for unweighted shortest paths (review)
- Dijkstra's algorithm with `heapq` for weighted shortest paths
- When to use BFS vs Dijkstra: complexity and problem constraints
- Route optimization in artist collaboration networks

Goals

- Understand when edge weights matter in graph problems
- Implement Dijkstra's algorithm using Python's `heapq`
- Build a weighted artist collaboration network (weight = collab count)
- Find shortest collaboration paths between artists
- Choose the right algorithm based on graph properties

Roadmap

First half (≈35 min)

- Motivation: unweighted vs weighted graphs
- BFS for unweighted shortest paths (quick review)
- Introducing edge weights: artist collaboration strength
- Dijkstra's algorithm: greedy shortest path
- Implementation with `heapq` priority queue
- In-class exercise 1 (commit required)

Second half (≈35 min)

- Building weighted artist collaboration networks
- Route optimization: finding strongest collaboration chains
- BFS vs Dijkstra: when to use each
- Complexity analysis: O(V + E) vs O((V + E) log V)
- Negative weights and why Dijkstra fails
- In-class exercise 2 (commit required)
- Complexity table and wrap-up

Setup: Load Spotify data

We'll build an artist collaboration network from track data.

In [ ]:

```
import csv
import heapq
from collections import defaultdict, deque

# Load artists
artists = {}
with open('data/artists.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        artists[row['artist_id']] = row

# Load tracks (normalize track_id)
tracks = {}
with open('data/tracks.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        tid = row['track_id'].strip().lower()
        row['track_id'] = tid
        # Join artist name from artists.csv
        aid = row.get('artist_id', '').strip()
        row['artist'] = artists.get(aid, {}).get('artist_name', 'Unknow
        tracks[tid] = row
```

```
print(f"Loaded {len(tracks)} tracks, {len(artists)} artists")
print(f"Sample track: {list(tracks.values())[0]}")
```

Part 1: Unweighted Graphs and BFS (Review)

Why shortest paths matter

Real-world problems:

- Social networks: degrees of separation
- Route planning: fewest stops, shortest distance
- Collaboration networks: shortest path between artists
- Dependency resolution: minimum build steps

Two cases:

1. **Unweighted graphs** — all edges cost 1 → use BFS
2. **Weighted graphs** — edges have different costs → use Dijkstra

BFS for unweighted shortest path

Key insight: BFS discovers nodes in increasing distance order.

Why it works:

- Process nodes level-by-level
- First time we reach a node = shortest path found
- All edges cost 1, so level = distance from start

Complexity: O(V + E) — visit each node once, check each edge once

In [ ]:

```
def bfs_shortest_path(graph, start, goal):
    """
    Find shortest path in unweighted graph using BFS.

    Args:
        graph: dict mapping node -> list of neighbors
        start: starting node
        goal: target node

    Returns:
        (distance, path) or (None, None) if unreachable
    """
    if start == goal:
        return (0, [start])

    queue = deque([start])
    discovered = {start}
    parent = {start: None}

    while queue:
        node = queue.popleft()

        for neighbor in graph.get(node, []):
            if neighbor not in discovered:
                discovered.add(neighbor)
                parent[neighbor] = node
                queue.append(neighbor)

                # Found goal - reconstruct path
                if neighbor == goal:
```

```
                    path = []
                    current = goal
                    while current is not None:
                        path.append(current)
                        current = parent[current]
                    path.reverse()
                    return (len(path) - 1, path)

    return (None, None)  # No path found
```

In [ ]:

```python
# Example: Simple collaboration network (unweighted)
# Each edge means "artists have collaborated"
simple_network = {
    'Taylor Swift': ['Ed Sheeran', 'Bon Iver'],
    'Ed Sheeran': ['Taylor Swift', 'Justin Bieber', 'Eminem'],
    'Bon Iver': ['Taylor Swift', 'Kanye West'],
    'Justin Bieber': ['Ed Sheeran', 'Post Malone'],
    'Eminem': ['Ed Sheeran', 'Rihanna'],
    'Kanye West': ['Bon Iver', 'Rihanna'],
    'Post Malone': ['Justin Bieber'],
    'Rihanna': ['Eminem', 'Kanye West']
}

# Shortest path from Taylor Swift to Rihanna
dist, path = bfs_shortest_path(simple_network, 'Taylor Swift', 'Rihanna
print(f"Distance: {dist} collaborations")
print(f"Path: {' → '.join(path)}")
# Output: Distance: 3 collaborations
#         Path: Taylor Swift → Ed Sheeran → Eminem → Rihanna
```

Problem: BFS ignores edge weights

What if collaborations have different strengths?

- 1 song together vs 10 songs together
- Route with more total collaborations might be "stronger"
- BFS only counts hops, not collaboration strength

Example:

```
Taylor --[1 collab]-- Ed --[1 collab]-- Rihanna    (BFS chooses this, total=2)
Taylor --[5 collabs]-- Bon Iver --[8 collabs]-- Kanye --[3 collabs]-- Rihanna (total=16!)
```

Need: Algorithm that considers edge weights → **Dijkstra's algorithm**

Part 2: Weighted Graphs

Representing weighted graphs

Adjacency list with weights:

```
# Instead of:
graph = {'A': ['B', 'C']}

# Use:
graph = {
    'A': [('B', 5), ('C', 3)],  # (neighbor, weight)
    'B': [('A', 5), ('D', 2)],
    'C': [('A', 3), ('D', 7)],
    'D': [('B', 2), ('C', 7)]
}
```

Weight interpretation:

- **Cost:** Distance, time, price (minimize)
- **Capacity:** Bandwidth, strength (maximize → negate weights)
- **Probability:** Likelihood (use log-transform)

In [ ]:

```python
# Build weighted collaboration network from tracks
# Weight = number of collaborations between artists
def build_weighted_network(tracks):
    """
    Build weighted artist collaboration graph.

    Edge weight = number of songs two artists collaborated on.

    Returns:
        dict mapping artist -> [(neighbor, weight), ...]
    """
    # Count collaborations
    collab_count = defaultdict(int)
    artist_collabs = defaultdict(set)

    for track in tracks.values():
        # 'artist' field may contain '&' or 'feat.' for collaborations
        import re
        artist_list = re.split(r'\s*&\s*|\s+feat\.\s*', track.get('artis
        artists = [a.strip() for a in artist_list if a.strip()]

        # For each pair of artists on this track
        for i in range(len(artists)):
            for j in range(i + 1, len(artists)):
                a1, a2 = sorted([artists[i], artists[j]])  # Canonical (
                collab_count[(a1, a2)] += 1
                artist_collabs[artists[i]].add(artists[j])
                artist_collabs[artists[j]].add(artists[i])

    # Build adjacency list with weights
```

```
    graph = defaultdict(list)
    for (a1, a2), count in collab_count.items():
        graph[a1].append((a2, count))
        graph[a2].append((a1, count))

    return graph

# For demonstration, create a small weighted example
weighted_network = {
    'Taylor Swift': [('Ed Sheeran', 1), ('Bon Iver', 5)],
    'Ed Sheeran': [('Taylor Swift', 1), ('Justin Bieber', 2), ('Eminem',
    'Bon Iver': [('Taylor Swift', 5), ('Kanye West', 8)],
    'Justin Bieber': [('Ed Sheeran', 2), ('Post Malone', 4)],
    'Eminem': [('Ed Sheeran', 3), ('Rihanna', 7)],
    'Kanye West': [('Bon Iver', 8), ('Rihanna', 3)],
    'Post Malone': [('Justin Bieber', 4)],
    'Rihanna': [('Eminem', 7), ('Kanye West', 3)]
}

print("Weighted collaboration network:")
for artist, neighbors in list(weighted_network.items())[:3]:
    print(f"{artist}: {neighbors}")
```

Part 3: Dijkstra's Algorithm

The greedy strategy

Idea: Always expand the closest undiscovered node.

Algorithm:

1. Start with distance 0 to source, ∞ to all others
2. Use a min-heap to track (distance, node) pairs
3. Pop the closest undiscovered node
4. Update distances to its neighbors (relaxation)
5. Repeat until we reach the goal or heap is empty

Key invariant: Once we pop a node from the heap, we've found its shortest path.

Why Dijkstra works

Greedy choice property:

- When we pop node `u` with distance `d`
- No shorter path to `u` can exist
- Why? Any other path goes through undiscovered nodes with distance ≥ d

Relaxation:

```
if dist[u] + weight(u, v) < dist[v]:
    dist[v] = dist[u] + weight(u, v)
    parent[v] = u
```

Requires: Non-negative edge weights (otherwise greedy fails)

In [ ]:

```
def dijkstra_shortest_path(graph, start, goal):
    """
    Find shortest path in weighted graph using Dijkstra's algorithm.

    Args:
        graph: dict mapping node -> [(neighbor, weight), ...]
        start: starting node
        goal: target node

    Returns:
        (distance, path) or (None, None) if unreachable
    """
    if start == goal:
        return (0, [start])

    # Initialize distances and parent pointers
    dist = {start: 0}
    parent = {start: None}

    # Min-heap: (distance, node)
    heap = [(0, start)]

    while heap:
        current_dist, node = heapq.heappop(heap)

        # Found goal - reconstruct path
        if node == goal:
            path = []
            current = goal
            while current is not None:
```

```
                path.append(current)
                current = parent[current]
            path.reverse()
            return (dist[goal], path)

        # Skip if we've already processed this node with a shorter path
        if current_dist > dist.get(node, float('inf')):
            continue

        # Relax edges to neighbors
        for neighbor, weight in graph.get(node, []):
            new_dist = current_dist + weight

            if new_dist < dist.get(neighbor, float('inf')):
                dist[neighbor] = new_dist
                parent[neighbor] = node
                heapq.heappush(heap, (new_dist, neighbor))

    return (None, None)  # No path found
```

In [ ]:

```
# Find shortest weighted path from Taylor Swift to Rihanna
dist, path = dijkstra_shortest_path(weighted_network, 'Taylor Swift', 'R

print(f"Shortest path (weighted):")
print(f"Total collaborations: {dist}")
print(f"Path: {' → '.join(path)}")

# Compare to BFS (unweighted)
simple_unweighted = {k: [n for n, w in v] for k, v in weighted_network.i
bfs_dist, bfs_path = bfs_shortest_path(simple_unweighted, 'Taylor Swift

print(f"\nBFS path (unweighted):")
print(f"Hops: {bfs_dist}")
print(f"Path: {' → '.join(bfs_path)}")
```

Dijkstra trace example

Finding shortest path: Taylor Swift → Rihanna

```
Initial:
  dist = {Taylor: 0, all others: ∞}
  heap = [(0, Taylor)]

Step 1: Pop Taylor (dist=0)
  Relax Ed Sheeran: 0 + 1 = 1
  Relax Bon Iver: 0 + 5 = 5
  heap = [(1, Ed), (5, Bon Iver)]

Step 2: Pop Ed Sheeran (dist=1)
  Relax Justin: 1 + 2 = 3
  Relax Eminem: 1 + 3 = 4
  heap = [(3, Justin), (4, Eminem), (5, Bon Iver)]

Step 3: Pop Justin (dist=3)
  Relax Post Malone: 3 + 4 = 7
  heap = [(4, Eminem), (5, Bon Iver), (7, Post)]

Step 4: Pop Eminem (dist=4)
  Relax Rihanna: 4 + 7 = 11
  heap = [(5, Bon Iver), (7, Post), (11, Rihanna)]

Step 5: Pop Bon Iver (dist=5)
  Relax Kanye: 5 + 8 = 13
  heap = [(7, Post), (11, Rihanna), (13, Kanye)]

Step 6: Pop Post Malone (dist=7)
  No improvement
```

Exercise 1: Implement Dijkstra

Task: Complete the `dijkstra_all_distances` function that computes shortest distances from a source to ALL nodes (not just one target).

Signature:

```
def dijkstra_all_distances(graph, start):
    """
    Compute shortest distances from start to all reachable nodes.

    Returns:
        dict mapping node -> shortest distance from start
    """
```

Hint: Don't stop at a goal — keep going until the heap is empty.

Test:

```
distances = dijkstra_all_distances(weighted_network, 'Taylor Swift')
print(distances['Rihanna'])  # Should print 11
print(distances['Post Malone'])  # Should print 7
```

Commit: `git add exercise1.py && git commit -m "Exercise 1: Dijkstra all distances"`

In [ ]:

```python
# Exercise 1 starter code
def dijkstra_all_distances(graph, start):
    """
    Compute shortest distances from start to all reachable nodes.

    Args:
        graph: dict mapping node -> [(neighbor, weight), ...]
        start: starting node

    Returns:
        dict mapping node -> shortest distance from start
    """
    dist = {start: 0}
    heap = [(0, start)]

    # TODO: Implement Dijkstra for all nodes
    # Keep going until heap is empty (don't stop at a goal)

    return dist
```

Exercise 1 Solution

In [ ]:

```python
def dijkstra_all_distances(graph, start):
    """
    Compute shortest distances from start to all reachable nodes.

    Args:
        graph: dict mapping node -> [(neighbor, weight), ...]
        start: starting node

    Returns:
        dict mapping node -> shortest distance from start
    """
    dist = {start: 0}
    heap = [(0, start)]

    while heap:
        current_dist, node = heapq.heappop(heap)

        # Skip if we've already processed this node
        if current_dist > dist.get(node, float('inf')):
            continue

        # Relax edges
        for neighbor, weight in graph.get(node, []):
            new_dist = current_dist + weight

            if new_dist < dist.get(neighbor, float('inf')):
```

```
                dist[neighbor] = new_dist
                heapq.heappush(heap, (new_dist, neighbor))

    return dist

# Test
distances = dijkstra_all_distances(weighted_network, 'Taylor Swift')
print("Distances from Taylor Swift:")
for artist in sorted(distances.keys()):
    print(f"  {artist}: {distances[artist]}")
```

Break (3 minutes)

Commit your Exercise 1 solution now!

When we return:

- Building weighted collaboration networks
- Route optimization applications
- BFS vs Dijkstra comparison
- Complexity analysis

Part 4: Route Optimization Applications

Building weighted collaboration networks

Real data: Count how many songs artists collaborated on

Two interpretations:

1. **Minimize hops** (minimize collaboration count) → use weights as-is
2. **Maximize strength** (prefer strong collaborations) → negate weights

Example:

- If Taylor and Ed have 10 collaborations, that's a "strong" connection
- To find strongest path: use weight = -10 (or 1/10 for "cost")
- Dijkstra finds minimum cost = maximum strength

In [ ]:

```python
# Build "strongest path" network (negate collaboration counts)
def strongest_path_network(weighted_graph):
    """
    Convert collaboration network to maximize strength.

    Original: high weight = many collaborations
    Negated: low weight = strong connection

    Returns:
        dict mapping artist -> [(neighbor, -weight), ...]
    """
    negated = {}
    for artist, neighbors in weighted_graph.items():
        negated[artist] = [(neighbor, -weight) for neighbor, weight in n
    return negated

# For positive weights, use reciprocal instead of negation
def strongest_path_network_reciprocal(weighted_graph):
    """
    Use reciprocal weights: 1/weight.

    High collaboration count → low cost.
    """
    reciprocal = {}
    for artist, neighbors in weighted_graph.items():
        reciprocal[artist] = [(neighbor, 1.0 / weight) for neighbor, wei
    return reciprocal

# Find strongest collaboration path
strongest = strongest_path_network_reciprocal(weighted_network)
```

```
dist, path = dijkstra_shortest_path(strongest, 'Taylor Swift', 'Rihanna

print(f"Strongest collaboration path:")
print(f"Cost (1/strength): {dist:.3f}")
print(f"Path: {' → '.join(path)}")
```

Application: Artist influence paths

Question: How are two artists connected through collaborations?

Use cases:

1. **Music recommendation:** Find artists similar via collaboration chains
2. **Influence analysis:** Trace how musical styles spread
3. **Network analysis:** Identify key connectors (high betweenness centrality)

Example:

```
Indie artist → Taylor Swift → Ed Sheeran → Justin Bieber → Pop mainstream
```

In [ ]:

```python
def find_influence_path(graph, artist1, artist2, max_hops=6):
    """
    Find shortest collaboration path between two artists.

    Args:
        graph: weighted collaboration network
        artist1: source artist
        artist2: target artist
        max_hops: maximum path length (avoid distant connections)

    Returns:
        (distance, path, total_collabs) or (None, None, None)
    """
    dist, path = dijkstra_shortest_path(graph, artist1, artist2)

    if dist is None or len(path) - 1 > max_hops:
        return (None, None, None)

    # Calculate total collaborations along path
    total_collabs = 0
    for i in range(len(path) - 1):
        artist_a = path[i]
        artist_b = path[i + 1]
        # Find weight between artist_a and artist_b
        for neighbor, weight in graph.get(artist_a, []):
            if neighbor == artist_b:
                total_collabs += weight
                break

    return (len(path) - 1, path, total_collabs)
```

```python
# Example usage
hops, path, collabs = find_influence_path(
    weighted_network,
    'Taylor Swift',
    'Post Malone'
)

if path:
    print(f"Influence path ({hops} degrees of separation):")
    print(f"Path: {' → '.join(path)}")
    print(f"Total collaborations: {collabs}")
else:
    print("No path found within max hops")
```

Part 5: BFS vs Dijkstra Comparison

When to use each algorithm

Property	BFS	Dijkstra
Graph type	Unweighted	Weighted (non-negative)
Time complexity	O(V + E)	O((V + E) log V)
Space complexity	O(V)	O(V)
Data structure	Queue (deque)	Min-heap (heapq)
Use case	Fewest hops	Minimum cost/distance
Implementation	Simple	Moderate

Rule of thumb:

- All edges equal cost? → **BFS**
- Different edge costs? → **Dijkstra**
- Negative weights? → Neither (use Bellman-Ford)

Complexity analysis

BFS: O(V + E)

- Visit each vertex once: O(V)
- Check each edge once: O(E)
- Queue operations: O(1) with deque

Dijkstra: O((V + E) log V)

- Each vertex processed once: O(V)
- Each edge relaxed once: O(E)
- Each heap push/pop: O(log V)
- Total heap operations: O((V + E) log V)

Why the log V factor?

- Binary heap operations (heappush/heappop) are O(log n)
- We may push up to E items into the heap
- But heap size never exceeds V active entries

In [ ]:

```python
# Empirical comparison: BFS vs Dijkstra
import time

def benchmark_shortest_path(graph, start, goal, weighted=False):
    """
    Benchmark BFS vs Dijkstra.

    Args:
        graph: either weighted or unweighted
        weighted: if True, use Dijkstra; else use BFS
    """
    start_time = time.perf_counter()

    if weighted:
        dist, path = dijkstra_shortest_path(graph, start, goal)
        algo = "Dijkstra"
    else:
        dist, path = bfs_shortest_path(graph, start, goal)
        algo = "BFS"

    elapsed = time.perf_counter() - start_time

    print(f"{algo}:")
    print(f"  Distance: {dist}")
    print(f"  Path length: {len(path) if path else 0}")
    print(f"  Time: {elapsed * 1000:.3f} ms")
    return elapsed

# Test on unweighted version
unweighted = {k: [n for n, w in v] for k, v in weighted_network.items()}
```

```
print("\nBenchmark: Taylor Swift → Rihanna\n")
bfs_time = benchmark_shortest_path(unweighted, 'Taylor Swift', 'Rihanna
print()
dijkstra_time = benchmark_shortest_path(weighted_network, 'Taylor Swift

print(f"\nDijkstra overhead: {dijkstra_time / bfs_time:.2f}x")
```

Why Dijkstra fails with negative weights

Problem: Greedy assumption breaks down.

Example:

```
A --[1]--> B --[-5]--> C
A --[3]----------> C
```

Dijkstra's trace:

1. Pop A (dist=0)
2. Relax B (dist=1), C (dist=3)
3. Pop B (dist=1) → already discovered C with dist=3
4. Relax C via B: dist = 1 + (-5) = -4 (better!)
5. **But C already popped!** Dijkstra won't revisit it.

Solution: Use Bellman-Ford (O(VE)) or detect negative cycles first.

In practice: Most real networks have non-negative weights.

Exercise 2: Collaboration path finder

Task: Build a collaboration path finder that:

1. Loads artist collaboration data from CSV
2. Finds the shortest path between two artists
3. Reports both hop count and total collaboration strength

CSV format (artists.csv):

```
artist1,artist2,song_count
Taylor Swift,Ed Sheeran,3
Ed Sheeran,Justin Bieber,2
```

Signature:

```python
def collaboration_path_finder(csv_path, artist1, artist2):
    """
    Find shortest collaboration path between two artists.

    Returns:
        (hops, path, total_songs) or (None, None, None)
    """
```

Test:

```
hops, path, songs = collaboration_path_finder(
    'data/artists.csv',
    'Taylor Swift',
    'Rihanna'
)
print(f"{hops} hops, {songs} total songs")
```

Commit: `git add exercise2.py && git commit -m "Exercise 2: Collaboration path finder"`

In [ ]:

```python
# Exercise 2 starter code
def load_collaboration_network(csv_path):
    """
    Load weighted artist collaboration network from CSV.

    CSV format: artist1,artist2,song_count

    Returns:
        dict mapping artist -> [(neighbor, weight), ...]
    """
    graph = defaultdict(list)

    # TODO: Load CSV and build graph
    # Remember: graph is undirected (add edge both ways)

    return graph

def collaboration_path_finder(csv_path, artist1, artist2):
    """
    Find shortest collaboration path between two artists.

    Args:
        csv_path: path to artists.csv
        artist1: source artist
        artist2: target artist

    Returns:
        (hops, path, total_songs) or (None, None, None)
    """
    graph = load_collaboration_network(csv_path)
```

```
    # TODO: Use Dijkstra to find shortest path
    # TODO: Calculate total song count along path

    return (None, None, None)
```

Exercise 2 Solution

In [ ]:

```python
def load_collaboration_network(csv_path):
    """
    Load weighted artist collaboration network from CSV.

    CSV format: artist1,artist2,song_count

    Returns:
        dict mapping artist -> [(neighbor, weight), ...]
    """
    graph = defaultdict(list)

    with open(csv_path) as f:
        reader = csv.DictReader(f)
        for row in reader:
            a1 = row['artist1']
            a2 = row['artist2']
            count = int(row['song_count'])

            # Undirected graph
            graph[a1].append((a2, count))
            graph[a2].append((a1, count))

    return graph

def collaboration_path_finder(csv_path, artist1, artist2):
    """
```

```
    Find shortest collaboration path between two artists.

    Args:
        csv_path: path to artists.csv
        artist1: source artist
        artist2: target artist

    Returns:
        (hops, path, total_songs) or (None, None, None)
    """
    graph = load_collaboration_network(csv_path)

    dist, path = dijkstra_shortest_path(graph, artist1, artist2)

    if dist is None:
        return (None, None, None)

    # Calculate total songs along path
    total_songs = 0
    for i in range(len(path) - 1):
        a = path[i]
        b = path[i + 1]
        for neighbor, weight in graph[a]:
            if neighbor == b:
                total_songs += weight
                break

    hops = len(path) - 1
    return (hops, path, total_songs)

# Test (if artists.csv exists)
```

```
# hops, path, songs = collaboration_path_finder(
#     'data/artists.csv',
#     'Taylor Swift',
#     'Rihanna'
# )
# if path:
#     print(f"Found path: {hops} hops, {songs} total songs")
#     print(f"Path: {' → '.join(path)}")
```

Wrap-up: Shortest paths in practice

You've learned:

- ✅ BFS finds shortest paths in unweighted graphs — O(V + E)
- ✅ Weighted graphs need different algorithms
- ✅ Dijkstra's algorithm uses a min-heap for greedy shortest paths
- ✅ Dijkstra runs in O((V + E) log V) with `heapq`
- ✅ Dijkstra requires non-negative edge weights
- ✅ Choose BFS for equal-cost edges, Dijkstra for weighted graphs
- ✅ Applied to Spotify: collaboration paths, influence analysis

Production tips:

- Use `heapq` for Dijkstra (stdlib, efficient)
- Track discovered to avoid revisiting (optimization)
- Consider early termination if only need one target
- For all-pairs shortest paths: run Dijkstra from each vertex

Next lecture: Greedy vs Exact Search

Don't forget: Commit both exercises before you leave!

Complexity table

Algorithm	Time	Space	Graph Type	Notes
BFS	O(V + E)	O(V)	Unweighted	Queue with deque
Dijkstra	O((V + E) log V)	O(V)	Weighted (≥0)	Min-heap with heapq
Bellman-Ford	O(VE)	O(V)	Any (detects -cycles)	Not covered
A*	O((V + E) log V)	O(V)	Weighted + heuristic	Not covered

Where:

- V = number of vertices (nodes)
- E = number of edges
- log V = logarithm base 2 (heap operations)

Next lecture preview

Lecture 16: Greedy vs Exact Search

- When greedy algorithms work (Dijkstra, Prim, Kruskal)
- When they fail (TSP, knapsack)
- Heuristic search and approximation algorithms
- Framing optimization problems

Homework:

- Review Dijkstra implementation
- Read about A* search (Dijkstra + heuristic)
- Think about graph problems in your domain

Project 3 (Rock Tour) due March 30

- Apply BFS/DFS to 3D maze traversal
- Graph modeling and implementation
- Start early — debugging graph algorithms takes time!